

Velocity Corporate Training Center, Pune

Transformers Notes

Transformers

SECTION 1: Motivation & Background

1. Why Transformers?
 2. Limitations of RNN, LSTM, GRU
 3. Encoder-Decoder Architecture in Seq2Seq
 4. Issues with RNN-based Encoder-Decoder (Bottleneck, Long-Term Dependencies)
 5. Need for Parallelization and Long-Range Attention
-

SECTION 2: Transformer Architecture Overview

1. Encoder-Decoder Structure
 2. Overview of all layers
 3. High-level Flow of Data
-

SECTION 3: Input Representation

1. Tokenization
 2. Embedding
 3. Positional Encoding (Sinusoidal or Learned)
-

SECTION 4: Self-Attention Mechanism

1. Concept of Attention
 2. Scaled Dot-Product Attention
 3. Multi-Head Attention
 4. Intuition Behind Multiple Heads
 5. Masking (Padding Mask & Look-Ahead Mask)
-

SECTION 5: Encoder Details

1. Input Embedding + Positional Encoding
 2. Multi-Head Self-Attention
 3. Add & Norm
 4. Feed Forward Neural Network (FFNN)
 5. Residual Connections
 6. Layer Normalization
-

SECTION 6: Decoder Details

1. Masked Multi-Head Self-Attention
 2. Encoder-Decoder Attention
 3. FFNN and Normalization
-

SECTION 7: Output Layer

1. Linear + Softmax
 2. Beam Search / Greedy Decoding
-

SECTION 8: Training Transformer

1. Loss Function (Cross Entropy)
 2. Label Smoothing
 3. Optimizer (Adam with Warm-up and LR Schedule)
 4. Regularization Techniques
-

SECTION 9: Applications & Evolution

- 1. Transformer in NLP: BERT, GPT, T5
- 2. Vision Transformers
- 3. Long-Sequence Transformers (Performer, Reformer, Longformer)

SECTION 10: Mathematical Intuition (Optional but Powerful)

- 1. Derivation of Scaled Dot-Product Attention
- 2. Matrix Shapes at Every Stage
- 3. Intuition Behind Scaling and Normalization

🧠 SECTION 1: Motivation & Background

1. Why Transformers?

Traditional models like **RNNs**, **LSTMs**, and **GRUs** process data sequentially — word by word — which makes them:

- **Slow** to train (no parallelization).
- **Weak** at capturing long-term dependencies (early words are "forgotten").
- **Difficult to optimize** for very long sequences.

✅ Transformers were introduced to **solve these issues using self-attention**: “*Attention is all you need*” — Vaswani et al., 2017

2. Limitations of RNN, LSTM, GRU

Limitation	RNN/LSTM/GRU Behavior
Sequential Processing	Cannot parallelize — processes one word at a time
Vanishing Gradients	Loses track of early input over long sequences
Fixed Context Window	Bottleneck when summarizing large context to a fixed-length vector
Slow Training	Backpropagation through time is computationally expensive

3. Encoder-Decoder Architecture in Seq2Seq

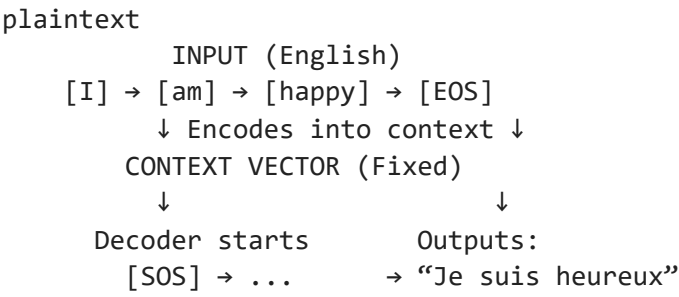
Before Transformers, **Sequence-to-Sequence (Seq2Seq)** models were used in tasks like **machine translation**.

🏗️ Architecture:

- **Encoder**: Takes input sequence and creates a context vector.
- **Decoder**: Generates output sequence using the context vector.

Example:

English Input: “I am happy.” French Output: “Je suis heureux.”



⚠️ Problems:

- Long sentences are **compressed into a single fixed-size vector** → **information loss**
- **Hard to learn long-term dependencies**

4. Need for Parallelization and Long-Range Attention

Transformers overcome RNN issues by:

- Using **self-attention** instead of recurrence
- Allowing **parallel processing of all words**
- Dynamically focusing on **important words** in the sentence, regardless of their position

Example:

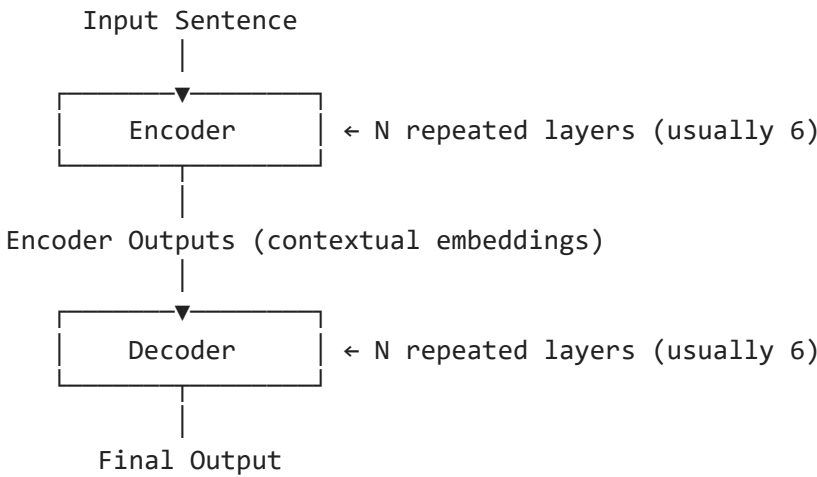
In the sentence: "*The animal didn't cross the street because it was tired.*"

- "It" should attend to "The animal" — **long-range dependency**.

SECTION 2: Transformer Architecture Overview

The **Transformer** is based on an **Encoder-Decoder architecture**, but unlike traditional Seq2Seq models, it does **not use recurrence or convolutions**. Instead, it relies entirely on **attention mechanisms**.

2.1 High-Level Architecture



2.2 Components of Transformer

Each encoder and decoder layer has **specific sublayers**:

Encoder Block:

Each of the N encoder layers consists of:

1. **Multi-Head Self-Attention**
2. **Feed Forward Neural Network (FFNN)**
3. **Add & Layer Normalization** after each of the above

Decoder Block:

Each of the N decoder layers consists of:

1. **Masked Multi-Head Self-Attention** (prevents peeking into the future)
2. **Encoder-Decoder Attention** (attends to encoder output)
3. **Feed Forward Neural Network (FFNN)**
4. **Add & Layer Normalization** after each block

2.3 Input & Output Flow

Encoder Side:

- Input tokens are converted to embeddings.
- **Positional encoding** is added to include word order.
- Multiple encoder layers apply self-attention to capture relationships.

Decoder Side:

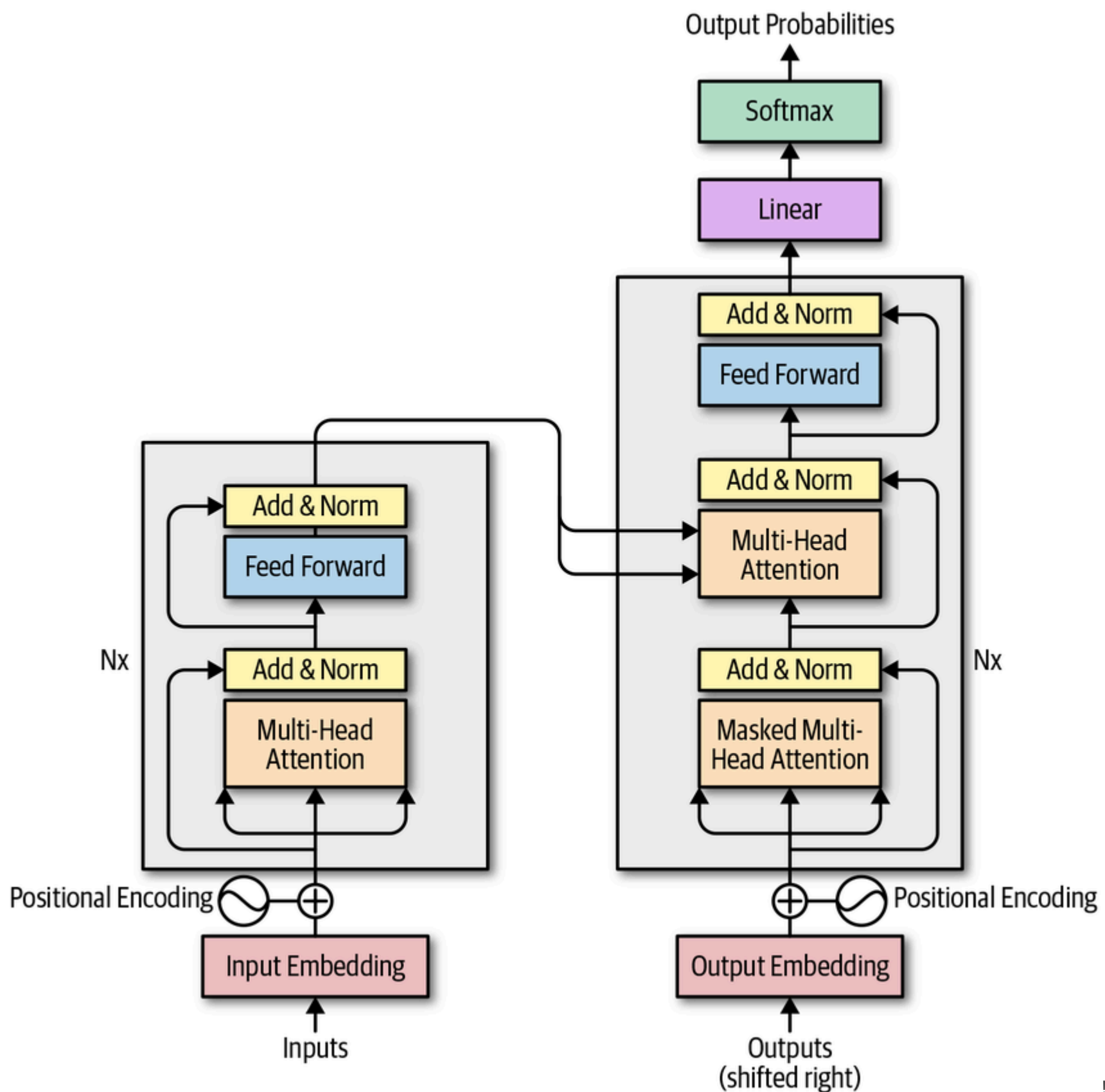
- Takes the **previous outputs** as input (teacher forcing during training).
- Uses **Masked Attention** to ensure only previous tokens are visible.
- **Attends to encoder output** using cross-attention.
- Outputs are passed through a final **linear + softmax** layer for prediction.

2.4 Summary of Flow

Stage	Description
Input	Token Embeddings + Positional Encoding
Encoder	Multi-head self-attention → FFNN (repeated N times)
Decoder	Masked self-attention → Encoder-decoder attention → FFNN (repeated N times)
Output Layer	Linear layer + Softmax for token prediction

✓ 2.5 Why This Architecture Works

- **Self-attention** enables the model to capture dependencies at any position.
- **Parallelization**: All tokens are processed simultaneously.
- **Multi-head** attention allows the model to focus on **multiple relationships** simultaneously.
- Residual + LayerNorm → Faster and more stable training.



■ SECTION 3: Input Representation

Before the Transformer model can understand any text, we need to **convert it into numerical format**. This is handled through three key steps:

✓ 3.1 Tokenization

◆ What is Tokenization?

It is the process of breaking down text into **smaller units (tokens)** — usually words or subwords.

Example: "Transformers are powerful." → Tokens: ["Transformers", "are", "powerful", "."]

◆ Types of Tokenization:

- **Word-level**: Simple but struggles with unknown words.
- **Subword-level** (e.g., BPE, WordPiece): Common in modern transformers (like GPT, BERT).
- **Character-level**: Rare in practice due to length.

◆ Vocabulary:

Each token is mapped to a unique integer using a vocabulary dictionary.

"Transformers are powerful." → [1045, 2071, 1996, 2295, 1012] (example IDs from BERT vocab)

✓ 3.2 Embedding

The tokens (now integers) are **passed into an embedding layer**, which converts each token into a **dense vector** (usually of size 512 or 768).

Example: Token ID 1045 → Embedding vector of shape (1, 512)

✦ These embeddings **capture the meaning** of the words (semantics) based on training.

◆ **Output Shape:**

If input has T tokens and model dimension is d_model (e.g., 512):

Output embedding shape: (T, d_model)

✓ 3.3 Positional Encoding

Transformers **do not have recurrence**, so they need a way to **understand the order of words**. That’s where **Positional Encoding** comes in.

◆ **Two Types:**

- **Sinusoidal Positional Encoding** (used in original paper)
- **Learned Positional Encoding** (learned during training)

◆ **Sinusoidal Formula:**

For a given position pos and dimension i :

$$\begin{aligned} \text{PE}(\text{pos}, 2i) &= \sin(\text{pos} / 10000^{(2i/d_model)}) \\ \text{PE}(\text{pos}, 2i+1) &= \cos(\text{pos} / 10000^{(2i/d_model)}) \end{aligned}$$

This creates **unique but smoothly varying vectors** for each position.

✓ 3.4 Final Input to Encoder

The final input passed to the encoder is:

$$\text{Input} = \text{TokenEmbedding} + \text{PositionalEncoding}$$

This results in a matrix of shape:

(sequence_length, d_model) Example: If we have 5 words and d_model = 512 → output = (5, 512)

✓ 3.5 Summary

Step	Purpose	Output Shape
Tokenization	Split sentence into tokens	(T,)
Embedding	Convert token ID to dense vector	(T, d_model)
Positional Encoding	Add position info to embeddings	(T, d_model)
Final Input	Embedding + Positional Encoding	(T, d_model)

SECTION 4: Self-Attention Mechanism

✓ 4.1 What is Attention?

Attention allows the model to **focus on different parts of the input** when processing a word.

Imagine reading: “The animal didn’t cross the road because it was tired.” We need to link “it” back to “animal”. Attention helps the model learn that relationship.

✓ 4.2 What is Self-Attention?

In **self-attention**, **every word attends to every other word in the same sequence**, including itself.

Example sentence: “The cat sat on the mat.” When encoding “cat”, the model also considers words like “sat” and “mat”.

✓ 4.3 Steps in Scaled Dot-Product Attention

We need **3 vectors** for each word:

- **Query (Q)**

- **Key (K)**
- **Value (V)**

These are obtained by multiplying the input embedding with learnable weight matrices:

$$\begin{aligned} Q &= X \times W_Q \\ K &= X \times W_K \\ V &= X \times W_V \end{aligned}$$

Then, we compute attention as:

$$\text{Attention}(Q, K, V) = \text{softmax}(Q \times K^T / \sqrt{d_k}) \times V$$

 Steps:

1. **Q × K^T**: Measure similarity between words.
2. **Scale** by $\sqrt{d_k}$ to avoid large dot products.
3. **Softmax**: Normalize the scores into probabilities.
4. **Multiply with V**: Weighted sum of values.

 4.4 Attention Score Example

Let's say we have a sentence of 3 tokens: ["I", "am", "happy"]

The attention for token **“am”**:

- Compare “am” with all tokens: “I”, “am”, “happy”
- Decide how much weight each word should get
- Compute new representation for “am” using weighted sum of all value vectors

 4.5 Multi-Head Attention

Instead of using just one attention function, we use **multiple heads**.

 Each head learns **different relationships** in the same sentence.

Steps:

- Split the embedding into **h** heads (e.g., 8)
- Apply self-attention independently
- Concatenate outputs
- Pass through a final projection matrix

$$\text{MultiHead}(Q, K, V) = \text{Concat}(\text{head}_1, \dots, \text{head}_h) \times W_O$$

This allows the model to learn multiple types of relationships simultaneously (e.g., subject-verb, adjective-noun).

 4.6 Masking in Attention

◆ **Padding Mask:**

- Masks out **<PAD>** tokens during attention computation.
- Prevents model from focusing on meaningless padding.

◆ **Look-Ahead Mask (in decoder):**

- Ensures a word can't see the **future words** during training.
- Used in language generation tasks.

 4.7 Final Output Shape

If:

- Input shape = (batch_size, seq_len, d_model)
- Number of heads = **h**
- Each head dimension = **d_k = d_model / h**

Then:

- Q, K, V → (batch_size, h, seq_len, d_k)
- Output of all heads concatenated → (batch_size, seq_len, d_model)

 4.8 Summary

Component	Description
Q, K, V	Created from input embeddings using learnable matrices
$Q \times K^T / \sqrt{d_k}$	Attention scores showing similarity
softmax	Converts scores to weights
Output	Weighted sum of V vectors
Multi-head	Learns multiple types of relationships
Masking	Prevents attending to padding or future words

Self-Attention in Transformers

1. What is Self-Attention?

Self-Attention is a mechanism where each word in a sequence learns to focus on other words **within the same sequence**, including itself, to build a **context-aware representation**.

It's like asking: **"While processing this word, which other words are important?"**

2. Why is Self-Attention Needed?

Traditional models (RNNs/LSTMs) compress the entire sentence into a fixed-size vector, which:

- **Loses long-term dependencies**
- **Struggles with parallelization**
- **Processes input sequentially**

Self-attention solves this by:

- Capturing relationships **regardless of distance**
- Processing **all words simultaneously**
- Generating better representations through **contextual weighting**

3. Key Components in Self-Attention

Each word is represented using 3 vectors derived from its embedding:

- **Query (Q)** – What the word is asking
- **Key (K)** – What each word offers
- **Value (V)** – The content each word contributes

These vectors are computed by multiplying the input embeddings X with trainable weight matrices:

$$\begin{aligned} Q &= X \times W_Q \\ K &= X \times W_K \\ V &= X \times W_V \end{aligned}$$

4. Attention Score Formula

Self-attention is computed using **Scaled Dot-Product Attention**:

$$\text{Attention}(Q, K, V) = \text{softmax}(Q \times K^T / \sqrt{d_k}) \times V$$

Steps:

1. Dot product of Q and K to get similarity scores.
2. Scale by $\sqrt{d_k}$ (dimension size).
3. Apply softmax to get attention weights.
4. Multiply weights by V to get the new representation.

5. Step-by-Step Numerical Example

Sentence: "Machine Learning Interview" Assume embedding size $d_{\text{model}} = 4$

Input Embeddings:

Token	Vector
Machine	[0.2, 0.1, 0.4, 0.3]
Learning	[0.1, 0.3, 0.2, 0.2]
Interview	[0.4, 0.2, 0.1, 0.1]

- Step 1: $Q = K = V = X$ (identity matrices assumed)
- Step 2: Compute $Q \times K^T$

	Machine	Learning	Interview
Machine	0.30	0.19	0.17
Learning	0.19	0.22	0.19
Interview	0.17	0.19	0.22

- Step 3: Scale by $\sqrt{4} = 2$

	Machine	Learning	Interview
Machine	0.15	0.095	0.085
Learning	0.095	0.11	0.095
Interview	0.085	0.095	0.11

- Step 4: Apply Softmax (row-wise)

Example for "Machine": $[0.15, 0.095, 0.085]$ Softmax $\approx [0.347, 0.328, 0.325]$

- Step 5: Weighted Sum of V

For "Machine":

```
Final vector ≈
0.347 × V[Machine] +
0.328 × V[Learning] +
0.325 × V[Interview]
≈ [0.232, 0.198, 0.237, 0.202]
```

This is the **contextualized embedding** for "Machine".

6. Interpretation of Output Vector

Example: "Machine" → $[0.232, 0.198, 0.237, 0.202]$

- This vector now **encodes not only the meaning of "Machine"**, but also its relationship to "Learning" and "Interview".
- It is **context-enriched**, not just a static embedding.
- Cannot be interpreted dimension-by-dimension — but it is useful for downstream tasks like classification, translation, etc.

7. Can We Tell How "Machine" is Related to "Interview"?

Yes — but not from the vector directly.

- How?

Method	How it helps
Attention Weights	Show how much "Machine" attends to "Interview"
Cosine Similarity	Quantifies similarity between their output vectors

For example, "Machine" → 32.5% attention to "Interview" means **strong contextual influence**.

8. Summary

Step	Description
Input	Embeddings for each token
Q, K, V	Derived using trainable matrices
Attention Scores	Dot product of $Q \times K^T$

Step	Description
Scaling	Divide by $\sqrt{d_k}$
Softmax	Normalize scores to get attention weights
Weighted Sum	Multiply weights with V to get new vector
Output	Contextualized embedding

Final Thought

Self-attention allows every token to learn ion:  **Section 5: Encoder Details** or a diagram of this process! ntation based on **contextual relevance**. It's the **foundation** of the Transformer's power.

Self-Attention in Transformers

✓ 1. What is Self-Attention?

Self-Attention is a mechanism where each word in a sequence learns to focus on other words **within the same sequence**, including itself, to build a **context-aware representation**.

It's like asking: **"While processing this word, which other words are important?"**

✓ 2. Why is Self-Attention Needed?

◆ Limitations of Traditional Models (RNNs, LSTMs, and Word Embeddings)

◆ A. Limitations of RNNs/LSTMs

- Loses Long-Term Dependencies** → Early words may be "forgotten" as sequences grow longer.
- Sequential Processing** → Words are processed one at a time — no parallelism.
- Fixed-Size Context Vector** → Entire sentence is compressed into a single vector, losing fine-grained relationships.

◆ B. Limitations of Traditional Word Embeddings (Word2Vec, GloVe)

- No Context Awareness** → Each word has a **single, static vector**, regardless of its meaning in different sentences.
 - "She sat on the **bank** of the river."
 - "He works at the **bank** downtown." → "Bank" has the same vector in both — leads to ambiguity.
- Same Representation in All Contexts**
 - "Model" in "train the **model**" vs. "fashion **model**" → Same vector.
- No Dynamic Relationship Building** → Can't adjust word meaning based on surrounding words or sentence structure.
- Fails with Polysemous Words** → One vector for words with multiple meanings: "light", "lead", "bat", "right" etc.

✓ How Self-Attention Solves These Problems:

Problem	How Self-Attention Helps
Long-term dependency loss	Words can attend to distant tokens
Sequential processing	All words processed in parallel
Context-insensitive embeddings	Contextualized representations per sentence
One meaning per word	Adjusts meaning based on context dynamically

✓ Real-World Sentence Examples

◆ Example 1:

"The girl who sat by the window in the red dress looked tired." ✓ "Girl" attends to "looked tired" by skipping irrelevant middle parts.

◆ Example 2:

"Although it was raining, the match continued." ✓ "Match" attends to "raining" to understand contrast.

- ◆ **Example 3:**

“Ravi gave Raj his bag.” ✓ “His” a” or “Raj” based on learned dependency.
- ◆ **Example 4:**

“Overfitting happens when a model performs well on training data but poorly on test data.” ✓ “Overfitting” attenining” and “test” to understand the concept.
- ◆ **Example 5:**

“Explain cross-validation and why it is important in model sele
- ◆ **Example 6:**

“He wrote the program in Python.” (programming language). “The python slithered across the forest floor.” (snake)
- ◆ **Example 7:**

“She bought an apple from the market.” (fruit). “Apple just released a new iPhone.” (company). ✓ Traditional embeddings give the same vector for both meanings.

✓

3. Key Components in Self-Attention

Each word is represented using 3 vectors derived from its embedding:

- **Query (Q)** – What the word is asking
- **Key (K)** – What each word offers
- **Value (V)** – The content each word contributes

These vectors are computed by multiplying the input embeddings `X` with trainable weight matrices:

$$Q = X \times W_Q$$
$$K = X \times W_K$$
$$V = X \times W_V$$

✓

5. Step-by-Step Numerical Example

Sentence: "Machine Learning Interview" Assume embedding size `d_model = 4`

- ◆ **Input Embeddings:**

Token	Vector
Machine	[0.2, 0.1, 0.4, 0.3]
Learning	[0.1, 0.3, 0.2, 0.2]
Interview	[0.4, 0.2, 0.1, 0.1]

- ◆ **Step 1: $Q = K = V = X$ (identity matrices assumed)**
- ◆ **Step 2: Compute $Q \times K^T$**

	Machine	Learning	Interview
Machine	0.30	0.19	0.17
Learning	0.19	0.22	0.19
Interview	0.17	0.19	0.22

- ◆ **Step 3: Scale by $\sqrt{4} = 2$**

	Machine	Learning	Interview
Machine	0.15	0.095	0.085
Learning	0.095	0.11	0.095
Interview	0.085	0.095	0.11

- ◆ **Step 4: Apply Softmax (row-wise)**

Example for "Machine": `[0.15, 0.095, 0.085]` Softmax $\approx$ `[0.347, 0.328, 0.325]`

- ◆ **Step 5: Weighted Sum of V**

For “Machine”:

```
Final vector ≈
0.347 × V[Machine] +
0.328 × V[Learning] +
0.325 × V[Interview]
≈ [0.232, 0.198, 0.237, 0.202]
```

This is the **contextualized embedding** for "Machine".

6. Interpretation of Output Vector

Example: "Machine" → [0.232, 0.198, 0.237, 0.202]

- This vector now **encodes not only the meaning of "Machine"**, but also its relationship to "Learning" and "Interview".
- It is **context-enriched**, not just a static embedding.
- Cannot be interpreted dimension-by-dimension — but it is useful for downstream tasks like classification, translation, etc.

7. Can We Tell How “Machine” is Related to “Interview”?

Yes — but not from the vector directly.

How?

Method	How it helps
Attention Weights	Show how much “Machine” attends to “Interview”
Cosine Similarity	Quantifies similarity between their output vectors

For example, "Machine" → 32.5% attention to "Interview" means **strong contextual influence**.

8. Summary

Step	Description
Input	Embeddings for each token
Q, K, V	Derived using trainable matrices
Attention Scores	Dot product of Q × K ^T
Scaling	Divide by √d _k
Softmax	Normalize scores to get attention weights
Weighted Sum	Multiply weights with V to get new vector
Output	Contextualized embedding

Final Thought

Self-attention allows every token to learn from the **entire sentence**, dynamically adjusting its representation based on **contextual relevance**. It's the **foundation** of the Transformer's power.

3. Key Components in Self-Attention: Why Q, K, V?

First, Let’s Revisit the Question:

We already have **word embeddings**, like [0.2, 0.1, 0.4, 0.3] for the word “Machine.” So why do we need to create **three different vectors — Query (Q), Key (K), and Value (V)** from this single embedding?

Answer:

Because in **self-attention**, every word plays **three different roles simultaneously**:

Role	Description
Query (Q)	The word is asking a question — “Whom should I pay attention to?”
Key (K)	The word is providing information — “Here’s what I can offer if you're interested.”
Value (V)	The word is sharing actual content — “Here’s my contribution if you decide to focus on me.”

Even though these roles are performed by the **same word**, they require **different representations** of the word embedding depending on the context.

✔ Real-World Intuition

Think of it like a **team meeting** where everyone is both:

- Asking questions,
- Responding to others,
- Contributing to the discussion.

So, every participant (word) needs to:

- Ask questions (**Query**) to others,
- Be discoverable with relevant answers (**Key**),
- Provide helpful content when selected (**Value**).

If everyone spoke with **the same tone, same intent, and same context**, the conversation would fail.

That’s why we need **Q, K, and V** — to capture these separate interactions.

✔ How It Works Mathematically

Let’s assume the input is a matrix **X** of word embeddings.

We generate:

$$Q = X \times W_Q \quad \# \text{ What each word is asking}$$
$$K = X \times W_K \quad \# \text{ What each word is offering}$$
$$V = X \times W_V \quad \# \text{ What each word will contribute}$$

Where:

- **X** is the input matrix (n_words × d_model)
- **W_Q**, **W_K**, **W_V** are learnable projection matrices (d_model × d_k)

Each word now gets **three different views** of itself:

- One for asking (Q)
- One for matching (K)
- One for sharing (V)

✔ Why Not Just Use the Embedding Directly?

If we used the same embedding for Q, K, and V:

- The model **could not distinguish between “who’s asking” and “who’s answering”**
- All interactions would be **symmetric and rigid**
- The model would **miss nuanced relationships**

By using **different linear projections**, we let the model:

- Decide **what to pay attention to**
- **Whom to attend to**
- **How much information to gather** from them

Analogy: Self-Attention as a Candidate Applying for Jobs

1234 Step	Real-World Candidate Process	Self-Attention Concept
Embedding	You have complete knowledge across many fields — Python, Art, Music, Data Science, Mechanical. ⚠️ But if you apply with your full profile everywhere, it causes confusion and poor alignment.	Word Embedding: The raw, static representation of the word with all meanings. ⚠️ Limitation: Lacks context, same vector in every sentence.
1 Query (Q)	You create a résumé focused on Data Science . You highlight only DS-related skills, tools, and projects.	Query: What the word is looking for or focusing on in other words
2 Key (K)	You visit Naukri or LinkedIn and view many job listings. Each job post describes what it is looking for (e.g., “must know Python, ML, statistics”)	Key: What each word offers to others — how it describes itself
3 Q × K^T (Scoring)	Your résumé is compared to each job listing . If your résumé matches the job's needs, it's a good match. A match score is generated.	Attention Score: Similarity between Query and Key — higher score = more attention
4 Softmax	You normalize the scores — maybe 70% match with job A, 20% with job B, 10% with job C	Softmax: Scores become attention weights — who to pay more or less attention to
5 Value (V)	You speak in an interview only about the things that matter to the job you're applying for (e.g., DS projects, not art/music)	Value: The actual content each word provides after being selected by attention

6

Output

You combine everything — your focused résumé, the job's requirements, and your relevant experiences — to determine your **final selection readiness**

Attention Output: A new context-aware vector is formed for each word

✓

Final Summary

Component	Role it Plays	Why It's Needed
Query (Q)	What the word wants to find	Helps focus on relevant words
Key (K)	What the word offers	Helps determine if it should be attended to
Value (V)	What the word actually provides	Supplies the content for the new representation

Without Q, K, and V, attention would be **static and context-blind**. With them, each word becomes a **dynamic participant**, adapting its behavior to every sentence.



✓

Example: Generating Q, K, V When Dimensions Are Different

◆

Input Embedding

Let's say the embedding for the word "Machine" is **3-dimensional**:

X = [0.4, 0.6, 0.8] # Shape: (1 × 3)

We now project it to a lower-dimensional space of **2**, using different **W_Q**, **W_K**, **W_V** matrices.

◆

Define Trainable Projection Matrices

Each matrix has shape: **W_Q**, **W_K**, **W_V** = (3 × 2)

W_Q = [[0.2, 0.8],
[0.5, 0.1],
[0.3, 0.4]]

W_K = [[0.6, 0.3],
[0.2, 0.9],
[0.7, 0.1]]

W_V = [[0.4, 0.2],
[0.1, 0.6],
[0.8, 0.5]]

✔ Step-by-Step Projection

1. Query (Q)

Q = X × W_Q
= [0.4, 0.6, 0.8] × $\begin{bmatrix} 0.2 & 0.8 \\ 0.5 & 0.1 \\ 0.3 & 0.4 \end{bmatrix}$
= [(0.4×0.2 + 0.6×0.5 + 0.8×0.3), (0.4×0.8 + 0.6×0.1 + 0.8×0.4)]
= [0.08 + 0.3 + 0.24, 0.32 + 0.06 + 0.32]
= [0.62, 0.70]

2. Key (K)

K = X × W_K
= [0.4, 0.6, 0.8] × $\begin{bmatrix} 0.6 & 0.3 \\ 0.2 & 0.9 \\ 0.7 & 0.1 \end{bmatrix}$
= [(0.4×0.6 + 0.6×0.2 + 0.8×0.7), (0.4×0.3 + 0.6×0.9 + 0.8×0.1)]
= [0.24 + 0.12 + 0.56, 0.12 + 0.54 + 0.08]
= [0.92, 0.74]

3. Value (V)

V = X × W_V
= [0.4, 0.6, 0.8] × $\begin{bmatrix} 0.4 & 0.2 \\ 0.1 & 0.6 \\ 0.8 & 0.5 \end{bmatrix}$
= [(0.4×0.4 + 0.6×0.1 + 0.8×0.8), (0.4×0.2 + 0.6×0.6 + 0.8×0.5)]
= [0.16 + 0.06 + 0.64, 0.08 + 0.36 + 0.4]
= [0.86, 0.84]

✔ Final Output Summary:

Vector	Meaning	Output (Dim 2)
Q	What "Machine" asks	[0.62, 0.70]
K	What "Machine" offers	[0.92, 0.74]
V	What "Machine" provides	[0.86, 0.84]

✔ Why This Matters

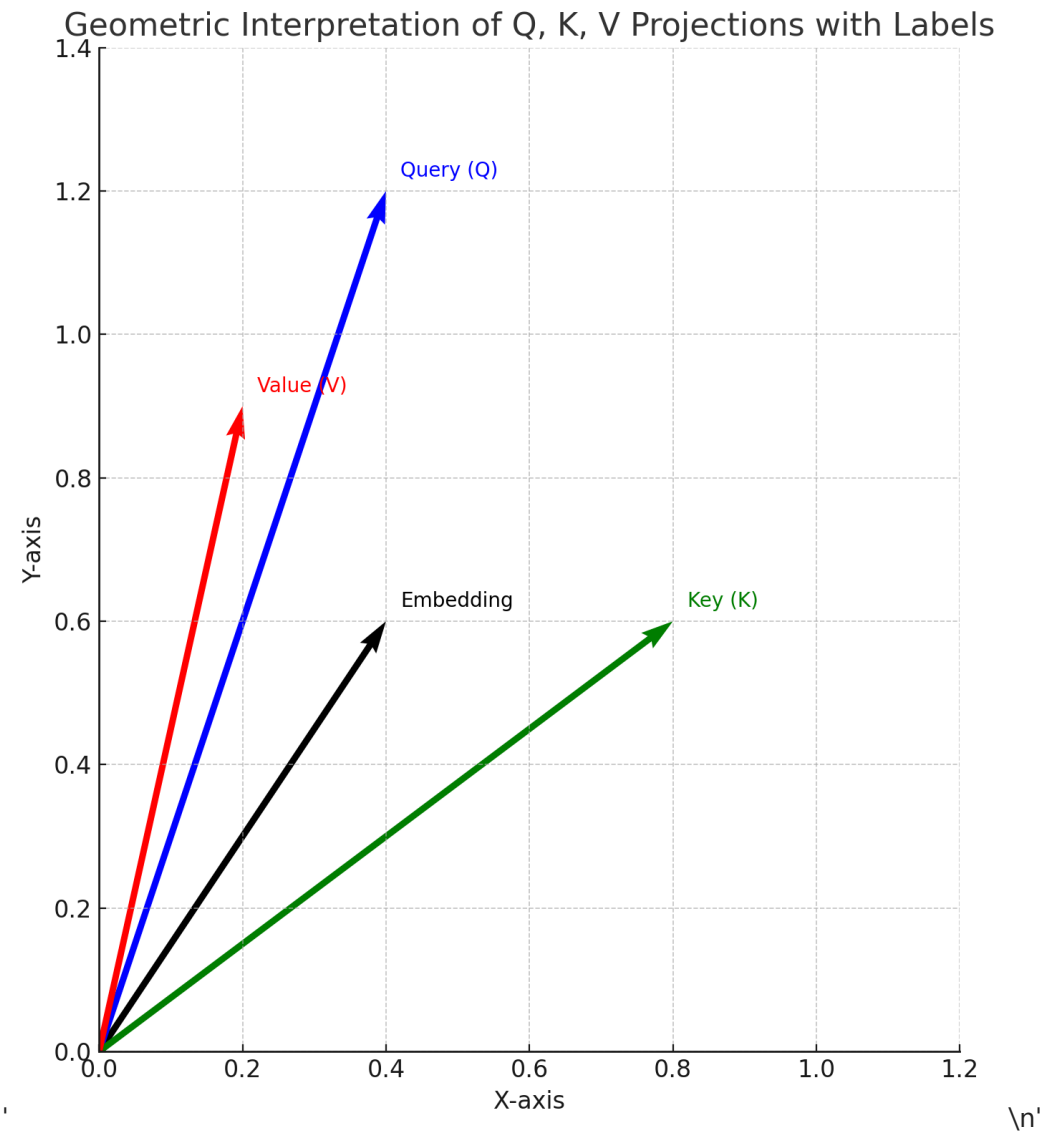
- The original input vector was **3-dimensional** (more expressive)
- Q, K, and V are **2-dimensional** (compressed for computation)
- Different projections allow the model to interpret the same word differently based on **task and context**
-

📌 Visual Intuition: Geometric View of Q, K, and V

To better understand the distinction between Query, Key, and Value vectors, refer to the **geometric visualization** iction.

- The image illustrates how a single embedding vector is **projected in different directions** to form **Q, K, and V because of linear transformation**.
- Each arrow represents a **different perspective** of the same word:
 - **Query**: What the word is looking for
 - **Key**: What the word offers to others
 - **Value**: What the word contributes when selected

📌 This geometric view reinforces the idea that **Q, K, and V are not redundant** — they are crucial for enabling **context-aware, dynamic attention** based on the role a word plays ine (Q × K^T) next. he heading structure.



✓ 4. Attention Score and Attention Weights

◆ Purpose

The **attention mechanism** determines **how much focus** a word should place on the other words in the sentence. This focus is calculated in two steps:

1. **Attention Score** (raw relevance measure between words)
2. **Attention Weights** (normalized scores that become actual weights)

◆ Step-by-Step Calculation with Example

We'll use the sentence:

"Machine Learning Interview"

We'll compute attention **for the word "Learning"**, using:

- Embedding dimension = 4
- Inputs: Q (for Learning), and K (for each word)

✓ Step 1: Define Query and Keys

Query vector for **Learning**:

$Q_{\text{Learning}} = [0.4, 0.3, 0.2, 0.1]$

Key vectors for:

- **Machine:** $[0.3, 0.2, 0.1, 0.1]$
- **Learning:** $[0.2, 0.3, 0.2, 0.2]$
- **Interview:** $[0.6, 0.4, 0.5, 0.4]$ ← Will receive highest attention

✓ Step 2: Compute Attention Scores ($Q \cdot K$)

This gives us raw similarity:

a. $Q \cdot K_{\text{Machine}}$

$$\begin{aligned} &= (0.4)(0.3) + (0.3)(0.2) + (0.2)(0.1) + (0.1)(0.1) \\ &= 0.12 + 0.06 + 0.02 + 0.01 = \boxed{0.21} \end{aligned}$$

b. $Q \cdot K_{\text{Learning}}$

= (0.4)(0.2) + (0.3)(0.3) + (0.2)(0.2) + (0.1)(0.2)

= 0.08 + 0.09 + 0.04 + 0.02 = 0.23

c. Q · K_Interview

= (0.4)(0.6) + (0.3)(0.4) + (0.2)(0.5) + (0.1)(0.4)

= 0.24 + 0.12 + 0.10 + 0.04 = 0.50 ✔ *Highest*

✔ Step 3: Scale by $\sqrt{d_k}$ (to avoid large gradients)

Scaled Score = $\frac{\text{Dot Product}}{\sqrt{4}} = \frac{\text{Score}}{2}$

So scaled scores become:

- Machine: $\frac{0.21}{2} = 0.105$
- Learning: $\frac{0.23}{2} = 0.115$
- Interview: $\frac{0.50}{2} = 0.25$

✔ Step 4: Apply Softmax

Convert the scaled scores into probabilities (attention weights).

a. Compute exponentials:

- $e^{0.105} \approx 1.110$
- $e^{0.115} \approx 1.122$
- $e^{0.25} \approx 1.284$

b. Total sum:

1.110 + 1.122 + 1.284 $\approx$ 3.516

c. Final Attention Weights:

- Machine: $\frac{1.110}{3.516} \approx 0.316$
- Learning: $\frac{1.122}{3.516} \approx 0.319$
- Interview: $\frac{1.284}{3.516} \approx 0.365$ ✔

✔ Final Result: Attention Weights

Word	Raw Score	Scaled	Softmax Weight
Machine	0.21	0.105	0.316
Learning	0.23	0.115	0.319
Interview	0.50	0.25	0.365 ✔

✔ Interpretation

- The word “**Learning**” is attending **most to “Interview”**.
- This means:
 - When generating the contextual representation of "Learning", it will give **36.5% weight to Interview**, more than Machine or itself.
- This could happen in a sentence like:

“The learning interview process was rigorous.” Where “Interview” is a strong context word for “Learning.”

✔ Conceptual Understanding

Concept	What It Means
Attention Score	Raw similarity between two words (how related they are)
Scaled Score	Helps avoid large values when dimensions are high
Softmax	Turns scores into a probability distribution
Attention Weight	Determines how much influence each word has in computing the new vector

✔ Weighted Sum of Value (V) — After Softmax Output

Once we’ve calculated **attention scores** and passed them through **softmax**, we get a set of **attention weights** for each word.

These weights determine **how much attention each word pays to all other words** in the sentence.

✔ What Happens Next?

We now compute the **weighted sum of the Value vectors (V)** based on the attention weights:

$$\text{Output}_{\text{Learning}} = 0.316 \cdot V_{\text{Machine}} + 0.319 \cdot V_{\text{Learning}} + 0.365 \cdot V_{\text{Interview}}$$

This is a **vector addition**, where each value vector is scaled by its attention weight and then summed element-wise.

✔ Final Output Vector (Example)

After calculation, we get:

[0.2779, 0.2362, 0.2273, 0.2414]

This is the **new embedding** for the word **“Learning”**, after attending to the entire sentence context.

✔ Interpretation of This Final Vector

- ♦ **What it is:**
 - A **context-aware representation** of the word “Learning”
 - It is a combination of what “Learning” **brings**, and what it **learned from “Machine” and “Interview”**
- ♦ **What it means:**
 - The model has decided that **“Interview” had the most relevant content** (based on the highest weight).
 - So the final vector is **influenced most** by the Value vector of **“Interview”**.
- ♦ **Why it matters:**
 - Unlike static embeddings (like Word2Vec), this vector is **dynamic** — it will **change based on the sentence**.
 - The same word “Learning” in a different sentence would have a **different vector**, because the attention weights would be different.

✔ Summary Table

Stage	Action		Output
Attention Weights	Softmax over scaled dot product of Q and K	0.316, 0.319, 0.365	
Weighted Sum of V	Multiply each Value vector by corresponding weight, then sum	Context-aware vector	
Final Output Vector	Contextual embedding for the word “Learning” based on attention to all words in the sentence	0.2779, 0.2362, 0.2273, 0.2414 ✔	

Scaling in Scaled Dot-Product Attention — Detailed Explanation

Context: What is Scaled Dot-Product Attention?

Recall the core attention calculation between Queries Q and Keys K :

$$\text{Scores} = Q \times K^T$$

These scores indicate how much each query attends to each key, computed as dot products of their vectors.

Why Do We Need Scaling?

Problem without scaling:

- The dot product between two vectors grows **larger in magnitude** as their dimension d_k increases.
- Specifically, the dot product’s **variance grows with d_k** (the dimension of the query/key vectors).
- When scores get large, the **softmax function saturates**, meaning:
 - Softmax outputs become very close to 0 or 1.
 - The gradient (derivative of softmax) becomes very small.
 - This leads to **vanishing gradients** during backpropagation.
 - Result: slower or unstable training.

Theoretical reasoning:

If each component of Q and K is roughly independent with mean 0 and variance 1, then:

- The dot product $Q \cdot K$ is a sum of d_k independent terms.
- By the **Central Limit Theorem**, the variance of the dot product scales as d_k .
- Hence, the raw dot product tends to have variance proportional to d_k .

Solution: Scale by $\frac{1}{\sqrt{d_k}}$

To keep the variance of dot products **approximately independent of d_k** , scale scores as:

$$\text{Scaled Scores} = \frac{Q \times K^T}{\sqrt{d_k}}$$

- This normalizes the magnitude, preventing very large values.
- Keeps softmax input values in a stable range for gradients.

Concrete Numeric Example (with dimension $d_k = 4$)

Suppose for a query Q and keys K_1, K_2, K_3 , raw dot products are:

$$\text{Raw Scores} = [20, 15, 10]$$

Without scaling, applying softmax:

$$\text{Softmax}(20, 15, 10) \approx [0.9999, 0.00005, 0.000005]$$

- Very peaky distribution: almost all attention on the first key.
- Gradients near zero for the smaller scores.

Now scale scores by:

$$\sqrt{d_k} = \sqrt{4} = 2$$

Scaled scores:

$$\frac{1}{2} \times [20, 15, 10] = [10, 7.5, 5]$$

Softmax on scaled scores:

$$\text{Softmax}(10, 7.5, 5) \approx [0.89, 0.09, 0.01]$$

- Much softer distribution.
- Allows gradients to flow better during training.
- Attention is still focused on the first key but less extreme.

Summary:

Without Scaling	With Scaling $\frac{1}{\sqrt{d_k}}$
Large scores cause softmax to saturate (close to 0 or 1)	Scores normalized, softmax outputs more balanced
Vanishing gradients during training	Better gradient flow, more stable training
Poor learning dynamics	Improved convergence speed and model stability

Intuition:

Scaling keeps the dot products in a "healthy" numerical range so softmax can properly **distribute attention** instead of putting all weight on a single token or none at all.

✔ Section 5: Multi-Head Attention (MHA)

◆ What is Multi-Head Attention?

Multi-Head Attention allows the model to **attend to information from multiple representation subspaces at different positions**.

Instead of applying a single self-attention mechanism, we **run multiple attention “heads” in parallel**, each with **different linear projections** of Q, K, and V.

◆ Why is Multi-Head Attention Needed?

✔ Limitation of Single Attention Head:

- A single attention head **only captures one type of relationship** or pattern between words.
- But natural language is complex — multiple relationships may exist:
 - Syntactic (structure)**
 - Semantic (meaning)**
 - Positional (relative position)**

✔ Multi-Head Solution:

- MHA enables the model to **capture diverse patterns** by learning from **different representation subspaces**.

Each head learns something different:

- One may focus on nearby words,
- One may focus on keywords,
- Another may attend to the subject of the sentence, etc.

✔ Why Multi-Head Attention is Needed – Real-World Examples Table

Scenario	Sentence	What Different Heads Learn	Why Multi-Head is Needed
Ambiguity: Instrument Use	She saw the man with the telescope.	Head 1: <i>man has telescope</i> Head 2: <i>she used telescope to see</i>	Disambiguates usage of “with the telescope”
Ambiguity: Modifiers	The girl saw the dog with the blue collar.	Head 1: <i>dog has collar</i> Head 2: <i>girl has collar</i>	Clarifies which noun “with the blue collar” modifies
Ambiguity: Apple Word	He ate the apple after using his Apple device.	Head 1: <i>apple</i> → fruit Head 2: <i>Apple</i> → tech brand	Resolves two meanings of “apple” based on context
Ambiguity: Animal or Language	He studies Python in the zoo.	Head 1: <i>Python</i> = animal Head 2: <i>Python</i> = programming language	Disambiguates the subject of study
Grammatical Structure	The engineer who fixed the bug explained the solution.	Head 1: Subject-verb → <i>engineer</i> → <i>explained</i> Head 2: <i>who fixed the bug</i> clause	Captures complex grammatical roles
Pronoun Resolution	Rahul built a model. He presented it at the conference.	Head 1: <i>He</i> refers to <i>Rahul</i> Head 2: <i>It</i> refers to <i>model</i>	Resolves different pronouns based on context
Data Science Terminology	The model was trained using decision trees and validated using cross-validation.	Head 1: <i>decision trees</i> → model Head 2: <i>cross-validation</i> → evaluation technique	Focuses separately on model type and evaluation method
Semantic Contrast	He works on Python and cares for endangered animals.	Head 1: <i>Python</i> = programming Head 2: <i>animals</i> = wildlife	Tracks two contrasting themes in the same sentence
Clause Resolution	Shreya gave the book to Rahul because he loves stories.	Head 1: <i>he</i> = Rahul Head 2: <i>book</i> = object of <i>gave</i>	Tracks co-reference and action-object relationships
Syntactic Roles	The cat chased the mouse and it escaped.	Head 1: <i>it</i> = cat Head 2: <i>it</i> = mouse	Resolves ambiguous pronoun based on sentence cues

✔ Why Multi-Head Attention is Needed – Real-World Examples Table

🧱 Architecture: How Multi-Head Attention Works

🔪 Step-by-Step Breakdown

Let’s assume:

- Input Embedding Dimension = `d_model = 512`
- Number of Heads = `h = 8`
- So, each head works with `d_k = d_model / h = 64`

✔ Step 1: Linear Projection of Input to Q, K, V for Each Head

For each input embedding **X**: We apply separate trainable weight matrices:

$$Q_i = X \cdot W_i^Q, \quad K_i = X \cdot W_i^K, \quad V_i = X \cdot W_i^V$$

For each **head i**, we learn:

- $W_i^Q \in \mathbb{R}^{d_{model} \times d_k}$
- $W_i^K \in \mathbb{R}^{d_{model} \times d_k}$
- $W_i^V \in \mathbb{R}^{d_{model} \times d_k}$

So now we have **h sets** of Q, K, and V.

✔ Step 2: Apply Scaled Dot-Product Attention in Each Head

$$\text{Attention}(Q_i, K_i, V_i) = \text{softmax} \left(\frac{Q_i \cdot K_i^\top}{\sqrt{d_k}} \right) \cdot V_i$$

This gives one **contextual output** for each head.

Each head captures **different relationships** between words.

✔ Step 3: Concatenate All Heads

We concatenate the output vectors from all heads:

$$\text{MultiHead}(Q, K, V) = \text{Concat}(\text{head}_1, \dots, \text{head}_h)$$

Resulting vector has shape:

$$(\text{batch_size}, \text{seq_len}, h \cdot d_k) = (\text{batch_size}, \text{seq_len}, d_{model})$$

✔ Step 4: Final Linear Transformation

$$\text{Final Output} = \text{Concat}(\text{head}_1, \dots, \text{head}_h) \cdot W^O$$

Where:

- $W^O \in \mathbb{R}^{(h \cdot d_k) \times d_{model}}$ is a learnable matrix
- This mixes information from all heads into a single output

🎯 Key Dimensions Summary (for d_model = 512, h = 8)

Component	Shape
Input Embedding	(batch_size, seq_len, 512)
W_Q, W_K, W_V	(512, 64) for each head
Q, K, V	(batch_size, seq_len, 64) × 8
Attention Output	(batch_size, seq_len, 64) × 8
Concat Heads	(batch_size, seq_len, 512)
Final Output	(batch_size, seq_len, 512)

🔍 Real-World Analogy

Multi-Head Attention = Multiple Interview Panels

Imagine you’re giving an interview with 8 different panels:

- Panel 1 checks your programming skills
- Panel 2 looks at your problem-solving
- Panel 3 reviews communication
- Panel 4 focuses on leadership

- ... and so on

Each panel (head) gives feedback from its unique lens. At the end, the HR team **merges all their feedback** to decide your final score.

Likewise, each attention head observes words from different perspectives, and the final output is a **rich, holistic representation**.

Interpretation

 Term	Meaning
Head	Independent self-attention mechanism
Q, K, V	Different projections to represent query, key, and value roles
Scaled Dot	Normalize dot product to prevent large gradients
Softmax	Converts scores to probabilities (attention weights)
Concat	Merge outputs from all heads
W ^o	Final linear layer to mix all heads

Summary: Why Multi-Head Attention Works

Benefit	Explanation
Parallelism	Multiple heads run simultaneously, enabling parallel processing
Diverse Representations	Each head learns unique patterns in the sentence
Improved Learning Capacity	Allows better generalization and richer feature extraction
Contextual Understanding	Captures both local and global dependencies across the sequence

--

Step	Operation & Explanation	Dimension (for “learning”)	Example Values / Calculations
1. Input Embedding X	Input embeddings for all words in sentence ("Machine learning interview")	3×4	$\begin{bmatrix} [0.2, 0.1, 0.3, 0.4], \\ [0.5, 0.3, 0.4, 0.2], \\ [0.1, 0.4, 0.2, 0.6] \end{bmatrix}$
2. Weight Matrix $W_Q^{(1)}$	Query projection matrix for Head 1 (trainable)	4×2	$\begin{bmatrix} 0.1 & 0.2 \\ 0.0 & 0.1 \\ 0.1 & 0.1 \\ 0.2 & 0.0 \end{bmatrix}$
3. Weight Matrix $W_K^{(1)}$	Key projection matrix for Head 1 (trainable)	4×2	$\begin{bmatrix} 0.2 & 0.1 \\ 0.1 & 0.2 \\ 0.0 & 0.1 \\ 0.1 & 0.1 \end{bmatrix}$
4. Weight Matrix $W_V^{(1)}$	Value projection matrix for Head 1 (trainable)	4×2	$\begin{bmatrix} 0.1 & 0.0 \\ 0.2 & 0.1 \\ 0.1 & 0.1 \\ 0.0 & 0.2 \end{bmatrix}$
5. Compute Query $Q^{(1)}$	Project “learning” embedding with $W_Q^{(1)}$: $Q = X \times W_Q^{(1)}$	1×2	$Q_{learning} = [0.13, 0.17]$
6. Compute Keys $K^{(1)}$	Project all embeddings with $W_K^{(1)}$: $K = X \times W_K^{(1)}$	3×2	$K_{machine} = [0.11, 0.18],$ $K_{learning} = [0.15, 0.18],$ $K_{interview} = [0.15, 0.24]$
7. Compute Values $V^{(1)}$	Project all embeddings with $W_V^{(1)}$: $V = X \times W_V^{(1)}$	3×2	$V_{machine} = [0.09, 0.06],$ $V_{learning} = [0.14, 0.12],$ $V_{interview} = [0.14, 0.14]$
8. Attention Scores	Calculate dot products $Q \cdot K^T$ for “learning” vs each key	1×3	Scores: $[0.0449, 0.0501, 0.0603]$

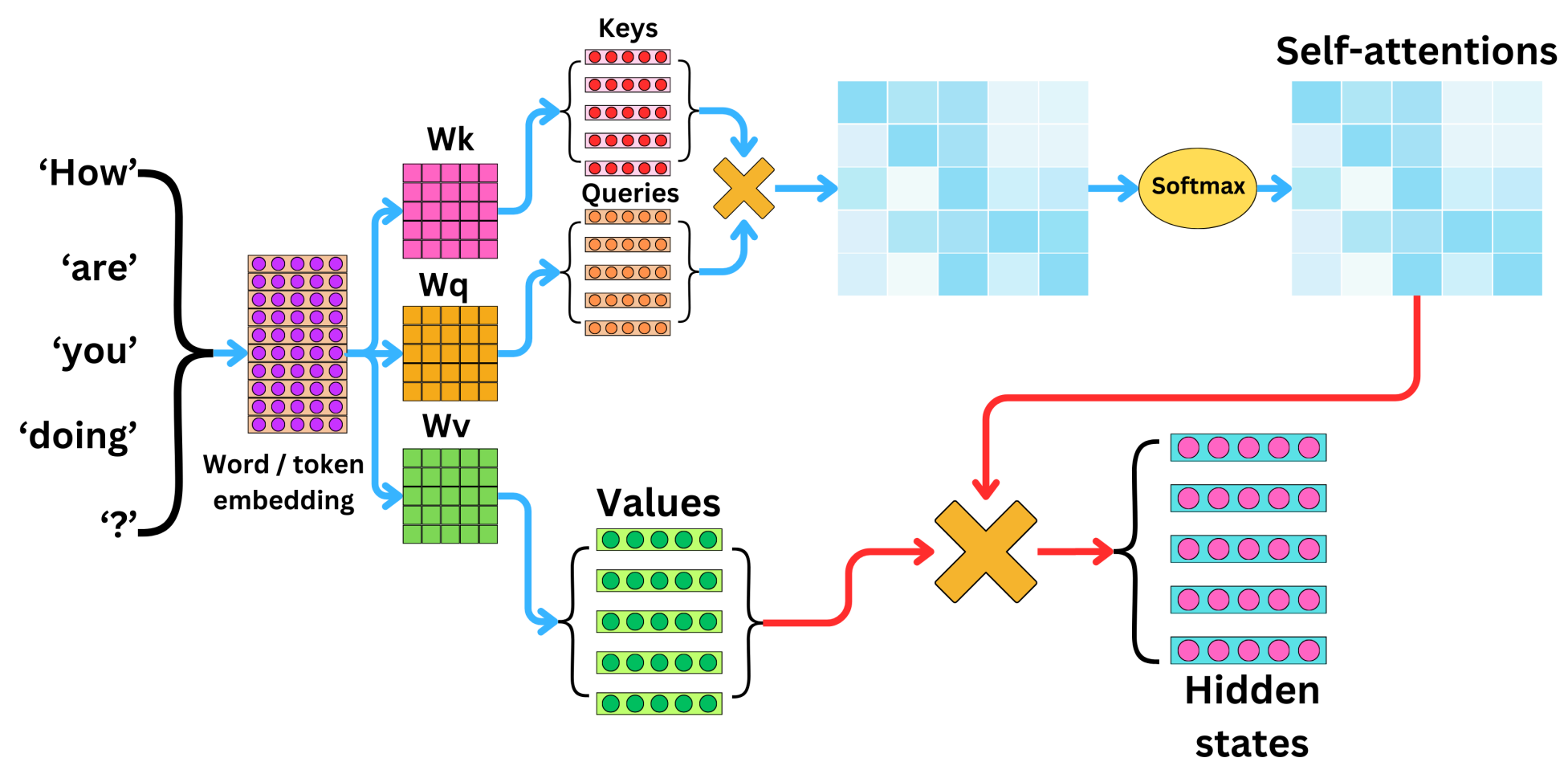
Step	Operation & Explanation	Dimension (for "learning")	Example Values / Calculations
9. Scale Scores	Scale scores by $\sqrt{d_k} = 1.41$	1×3	Scaled: $[0.0318, 0.0355, 0.0428]$
10. Softmax	Apply softmax to get attention weights	1×3	Weights: $[0.3316, 0.3329, 0.3355]$
11. Weighted Sum of $V^{(1)}$	Multiply each $V^{(1)}$ by corresponding weight and sum to get output vector for Head 1	1×2	Output Head 1: $[0.1234, 0.1068]$
12. Repeat for Head 2	Repeat steps 2–11 for Head 2 using different weight matrices $W_Q^{(2)}, W_K^{(2)}, W_V^{(2)}$	1×2	Assume Output Head 2: $[0.10, 0.15]$
13. Concatenate Heads	Concatenate outputs from Head 1 and Head 2 to get combined vector	1×4	Concatenated: $[0.1234, 0.1068, 0.10, 0.15]$

| 14. Output Projection W_O | Multiply concatenated vector by output projection matrix $W_O \in \mathbb{R}^{4 \times 4}$ to return to model dimension $d_{model} = 4$ | 1×4 |

Example W_O :

$$\begin{bmatrix} 0.2 & 0.1 & 0.3 & 0.4 \\ 0.1 & 0.2 & 0.2 & 0.1 \\ 0.3 & 0.4 & 0.1 & 0.2 \\ 0.2 & 0.3 & 0.4 & 0.1 \end{bmatrix}$$

Final output: multiply and get \$1 sy-to-copy code block!



Detailed Multi-Head Attention Dimension Flow

Sentence: Velocity corporate training center Word: corporate

Setup:

- **Model embedding size:** $d_{model} = 512$
- **Number of heads:** $h = 8$
- **Dimension per head:** $d_k = d_v = \frac{512}{8} = 64$
- **Sequence length:** $seq_len = 4$ (since the sentence has 4 words)
- **Batch size:** 1 (single sentence)

Step 1: Input Embedding of "corporate"

- The input word "corporate" is represented as a vector of size 512.

- Shape:

$$(1 \times 512)$$

- This embedding combines token and positional information.

Step 2: Linear Projections to Queries, Keys, and Values

- For each of the 8 heads, the embedding vector is linearly transformed into queries (Q), keys (K), and values (V) using trainable weight matrices:

$$W_Q, W_K, W_V \in \mathbb{R}^{512 \times 64}$$

- Resulting vectors for "corporate" per head:

$$Q, K, V \in \mathbb{R}^{1 \times 64}$$

Step 3: Compute Attention Scores

- The query vector for "corporate" is compared against key vectors of **all words** in the sequence (Velocity, corporate, training, center) to compute similarity scores:

$$\text{Scores} = Q \times K^T \in \mathbb{R}^{1 \times 4}$$

- This gives how much "corporate" attends to each word in the sentence.

Step 4: Scale and Apply Softmax

- To stabilize gradients, scores are scaled by dividing by $\sqrt{64} = 8$.
- Softmax normalizes the scaled scores into attention weights (probabilities) over the 4 tokens:

$$\text{Attention Weights} \in \mathbb{R}^{1 \times 4}$$

Step 5: Compute Weighted Sum of Values

- Attention weights are used to take a weighted sum of value vectors of all tokens:

$$\text{Output per head} = \text{Attention Weights} \times V \in \mathbb{R}^{1 \times 64}$$

- This produces the output vector for "corporate" from one attention head.

Step 6: Concatenate All Heads

- Outputs from all 8 heads (each of size 1×64) are concatenated:

$$\text{Concatenated Output} \in \mathbb{R}^{1 \times 512}$$

Step 7: Final Linear Projection

- The concatenated vector is multiplied by the output projection matrix $W_O \in \mathbb{R}^{512 \times 512}$ to combine information across heads and restore embedding dimension:

$$\text{Final Output} \in \mathbb{R}^{1 \times 512}$$

- This is the context-aware embedding of the word "corporate" after attention.

Summary Table

Step	Dimension	Description
Input Embedding	1×512	Original embedding of "corporate"
Project Q, K, V	1×64 per head	Linear projections for each of 8 heads
Attention Scores	1×4	Scores comparing "corporate" to all 4 tokens
Attention Weights	1×4	Softmax-normalized attention weights
Weighted Sum of Values	1×64 per head	Weighted sum of values for each head

Step	Dimension	Description
Concatenate Heads	1×512	Concatenate outputs of 8 heads
Final Output Projection	1×512	Output projection to combine heads and produce final output

Layer Normalization in Transformers

What is Layer Normalization?

- Layer Normalization (LayerNorm)** is a technique to **normalize the inputs of a layer** across the features (embedding dimensions) for each data point (token) individually.
- It ensures the **mean is 0 and variance is 1** for the features of each token embedding vector.
- Unlike Batch Normalization, which normalizes across a batch, LayerNorm normalizes **within each sample independently**.

Why LayerNorm?

- It **stabilizes and speeds up training** by reducing internal covariate shift.
- It helps the model **learn faster and generalize better**.
- Important for **deep models** like Transformers to avoid exploding or vanishing gradients.

Where is LayerNorm Used in Transformer?

1. After Multi-Head Attention:

The output of the Multi-Head Attention sub-layer is **added to the original input (residual connection)**, and then **LayerNorm is applied** on the sum:

$$\text{Norm}_1 = \text{LayerNorm}(X + \text{MultiHeadAttention}(X))$$

2. After Feed Forward Network (FFN):

Similarly, the output of the FFN sub-layer is added to its input, and then LayerNorm is applied:

$$\text{Norm}_2 = \text{LayerNorm}(\text{Norm}_1 + \text{FFN}(\text{Norm}_1))$$

How LayerNorm Works (Mathematically)

For each token embedding vector $\mathbf{x} \in \mathbb{R}^{d_{model}}$:

1. Compute mean and variance:

$$\mu = \frac{1}{d_{model}} \sum_{i=1}^{d_{model}} x_i$$
$$\sigma^2 = \frac{1}{d_{model}} \sum_{i=1}^{d_{model}} (x_i - \mu)^2$$

2. Normalize:

$$\hat{x}_i = \frac{x_i - \mu}{\sqrt{\sigma^2 + \epsilon}}$$

where ϵ is a small number to avoid division by zero.

3. Scale and shift (learnable parameters):

$$y_i = \gamma_i \hat{x}_i + \beta_i$$

where γ and β are vectors of parameters learned during training (dimension d_{model}).

Role of LayerNorm in Transformer Block

- Normalizes the summed vectors (input + sublayer output)** so the distribution remains stable.
- Works hand-in-hand with **residual connections** to allow **better gradient flow**.
- Improves **training speed and model convergence**.

Summary in Transformer Block

Input X
↓
Multi-Head Attention
↓

Add & Norm $\leftarrow (X + \text{Multi-Head Attention output}) + \text{LayerNorm}$
↓
Feed Forward Network (FFN)
↓
Add & Norm $\leftarrow (\text{Previous output} + \text{FFN output}) + \text{LayerNorm}$
↓
Output to next block or final layer

Example: One Token Embedding (4 dimensions) going through Attention + LayerNorm + FFN + LayerNorm

Initial token embedding vector (input):

$$x = [0.1, -0.2, 0.4, 0.5]$$

Step 1: Multi-head Self-Attention output for this token

Assume after multi-head attention, the output vector for this token is:

$$\text{AttentionOutput} = [0.2, -0.1, 0.3, 0.7]$$

Step 2: Add residual connection

Add the original input x to the attention output element-wise:

$$z = x + \text{AttentionOutput} = [0.1 + 0.2, -0.2 + (-0.1), 0.4 + 0.3, 0.5 + 0.7] = [0.3, -0.3, 0.7, 1.2]$$

Step 3: Apply Layer Normalization on vector z

Calculate mean and variance of z :

$$\mu_z = \frac{0.3 + (-0.3) + 0.7 + 1.2}{4} = \frac{1.9}{4} = 0.475$$
$$\sigma_z^2 = \frac{(0.3 - 0.475)^2 + (-0.3 - 0.475)^2 + (0.7 - 0.475)^2 + (1.2 - 0.475)^2}{4}$$

Calculate each term:

- $(0.3 - 0.475)^2 = (-0.175)^2 = 0.0306$
- $(-0.3 - 0.475)^2 = (-0.775)^2 = 0.6006$
- $(0.7 - 0.475)^2 = 0.225^2 = 0.0506$
- $(1.2 - 0.475)^2 = 0.725^2 = 0.5256$

Sum = $0.0306 + 0.6006 + 0.0506 + 0.5256 = 1.2074$

Divide by 4:

$$\sigma_z^2 = \frac{1.2074}{4} = 0.3019$$

Normalize each element:

Using $\epsilon = 10^{-5}$,

$$\hat{z}_i = \frac{z_i - \mu_z}{\sqrt{\sigma_z^2 + \epsilon}} = \frac{z_i - 0.475}{\sqrt{0.3019 + 0.00001}} = \frac{z_i - 0.475}{0.5495}$$

Calculate each normalized value:

- $\hat{z}_1 = \frac{0.3 - 0.475}{0.5495} = \frac{-0.175}{0.5495} \approx -0.318$
- $\hat{z}_2 = \frac{-0.3 - 0.475}{0.5495} = \frac{-0.775}{0.5495} \approx -1.409$
- $\hat{z}_3 = \frac{0.7 - 0.475}{0.5495} = \frac{0.225}{0.5495} \approx 0.409$
- $\hat{z}_4 = \frac{1.2 - 0.475}{0.5495} = \frac{0.725}{0.5495} \approx 1.320$

Step 4: Scale and shift (learnable parameters γ, β)

Assuming

$$\gamma = [1, 1, 1, 1], \quad \beta = [0, 0, 0, 0]$$

Output of first LayerNorm:

$$y = \gamma \odot \hat{z} + \beta = [-0.318, -1.409, 0.409, 1.320]$$

Step 5: Feed Forward Neural Network (FFN) layer

Suppose FFN transforms this vector to:

$$\text{FFNOutput} = [0.5, -0.6, 0.1, 1.0]$$

Step 6: Add residual connection again

Add input of FFN layer (which is y) to FFN output:

$$s = y + \text{FFNOutput} = [-0.318 + 0.5, -1.409 + (-0.6), 0.409 + 0.1, 1.320 + 1.0] = [0.182, -2.009, 0.509, 2.320]$$

Step 7: Apply Layer Normalization again on s

Calculate mean and variance of s :

$$\mu_s = \frac{0.182 + (-2.009) + 0.509 + 2.320}{4} = \frac{1.002}{4} = 0.2505$$
$$\sigma_s^2 = \frac{(0.182 - 0.2505)^2 + (-2.009 - 0.2505)^2 + (0.509 - 0.2505)^2 + (2.320 - 0.2505)^2}{4}$$

Calculate each:

- $(0.182 - 0.2505)^2 = (-0.0685)^2 = 0.00469$
- $(-2.009 - 0.2505)^2 = (-2.2595)^2 = 5.1062$
- $(0.509 - 0.2505)^2 = 0.2585^2 = 0.06684$
- $(2.320 - 0.2505)^2 = 2.0695^2 = 4.2848$

$$\text{Sum} = 0.00469 + 5.1062 + 0.06684 + 4.2848 = 9.4625$$

Divide by 4:

$$\sigma_s^2 = \frac{9.4625}{4} = 2.3656$$

Normalize s :

$$\hat{s}_i = \frac{s_i - \mu_s}{\sqrt{\sigma_s^2 + \epsilon}} = \frac{s_i - 0.2505}{\sqrt{2.3656 + 0.00001}} = \frac{s_i - 0.2505}{1.5384}$$

Calculate each normalized value:

- $\hat{s}_1 = \frac{0.182 - 0.2505}{1.5384} = \frac{-0.0685}{1.5384} \approx -0.0445$
 - $\hat{s}_2 = \frac{-2.009 - 0.2505}{1.5384} = \frac{-2.2595}{1.5384} \approx -1.469$
 - $\hat{s}_3 = \frac{0.509 - 0.2505}{1.5384} = \frac{0.2585}{1.5384} \approx 0.168$
 - $\hat{s}_4 = \frac{2.320 - 0.2505}{1.5384} = \frac{2.0695}{1.5384} \approx 1.345$
-

Final output of this Transformer encoder block for the token embedding is:

$$[-0.0445, -1.469, 0.168, 1.345]$$

Summary:

Step	Vector
Input embedding x	[0.1, -0.2, 0.4, 0.5]
Attention output	[0.2, -0.1, 0.3, 0.7]
Residual + Attention	[0.3, -0.3, 0.7, 1.2]
LayerNorm after Attention	[-0.318, -1.409, 0.409, 1.320]
FFN output	[0.5, -0.6, 0.1, 1.0]
Residual + FFN	[0.182, -2.009, 0.509, 2.320]
LayerNorm after FFN	[-0.0445, -1.469, 0.168, 1.345]

This is how **Layer Normalization stabilizes and normalizes outputs after both attention and feed-forward layers**, with residual connections helping to preserve original input info.